

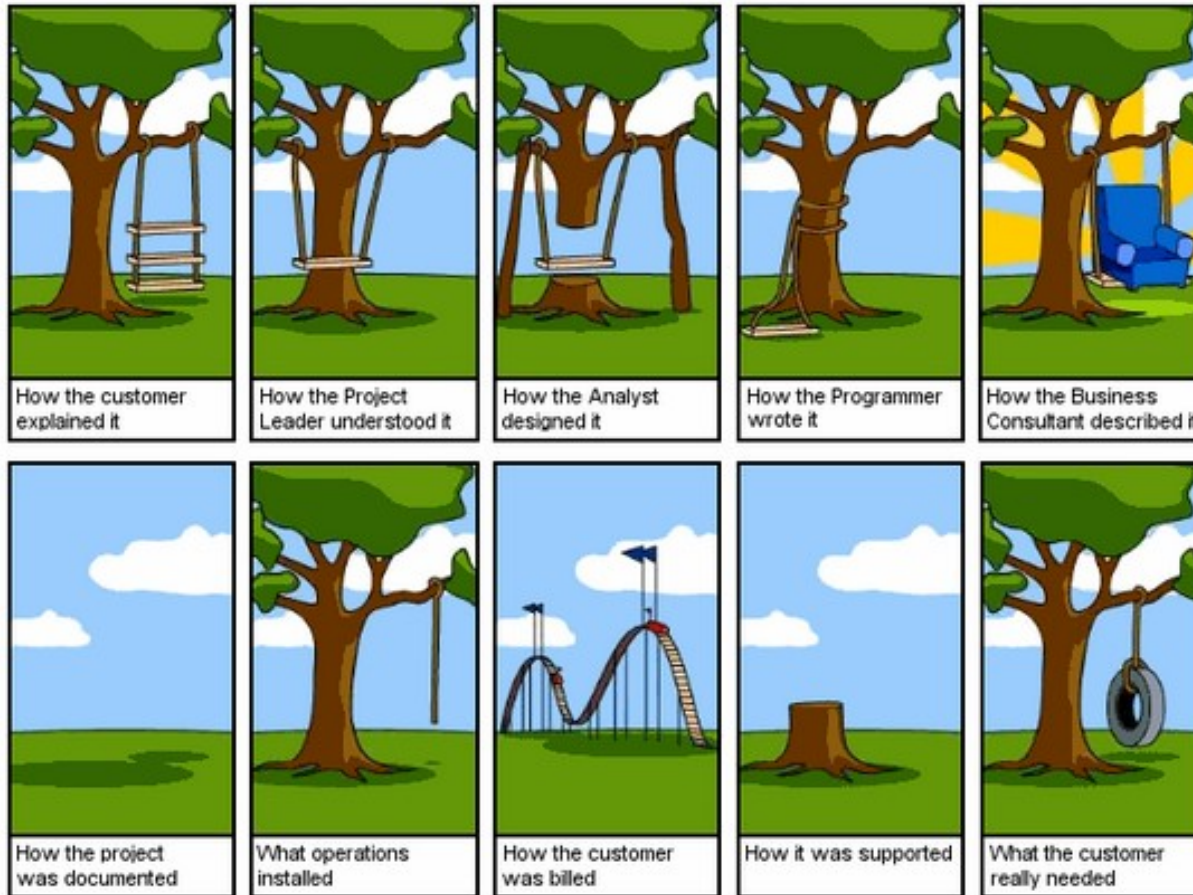
CSSE3113 – Introduction to Software Engineering

Project Management Concepts

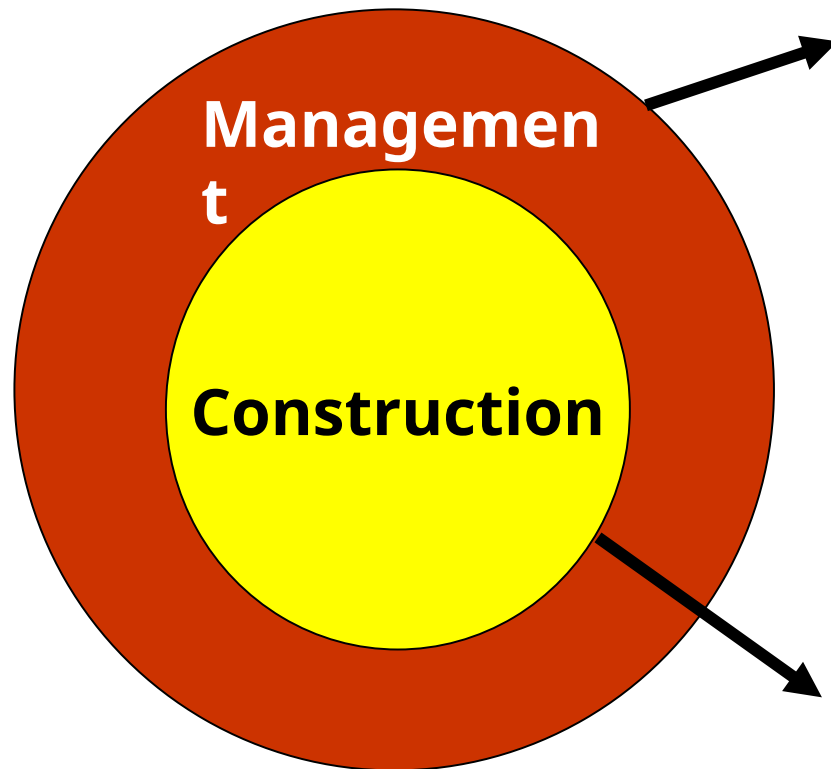
Why Software Fails?

- Failures occur because:
 - of the size of the project relative to previous efforts
 - key personnel have left
 - of failure to understand requirements
 - the project delivers, but lacks the required quality
 - of the introduction of new technology
 - of many, many other reasons

The dilemma



Software Development



Project planning and management
Configuration management
Quality assurance
Installation and training
etc.

Requirements
Design
Coding
Testing
Maintenance **etc.**

What is a Project?

- A **temporary endeavor** undertaken to create a unique product, service or result.
- Definite beginning and a definite end
- Does not mean short duration
- Produce a product or artifact, quantifiable, an end item or a component item
- A capability to perform a service, such as call center
- A result, such as research results or documents.

What is Management?

Management can be defined as **all activities and tasks undertaken by one or more persons** for the purpose of **planning and controlling the activities** of others in order **to achieve objectives** or complete an activity that could not be achieved by other acting independently.

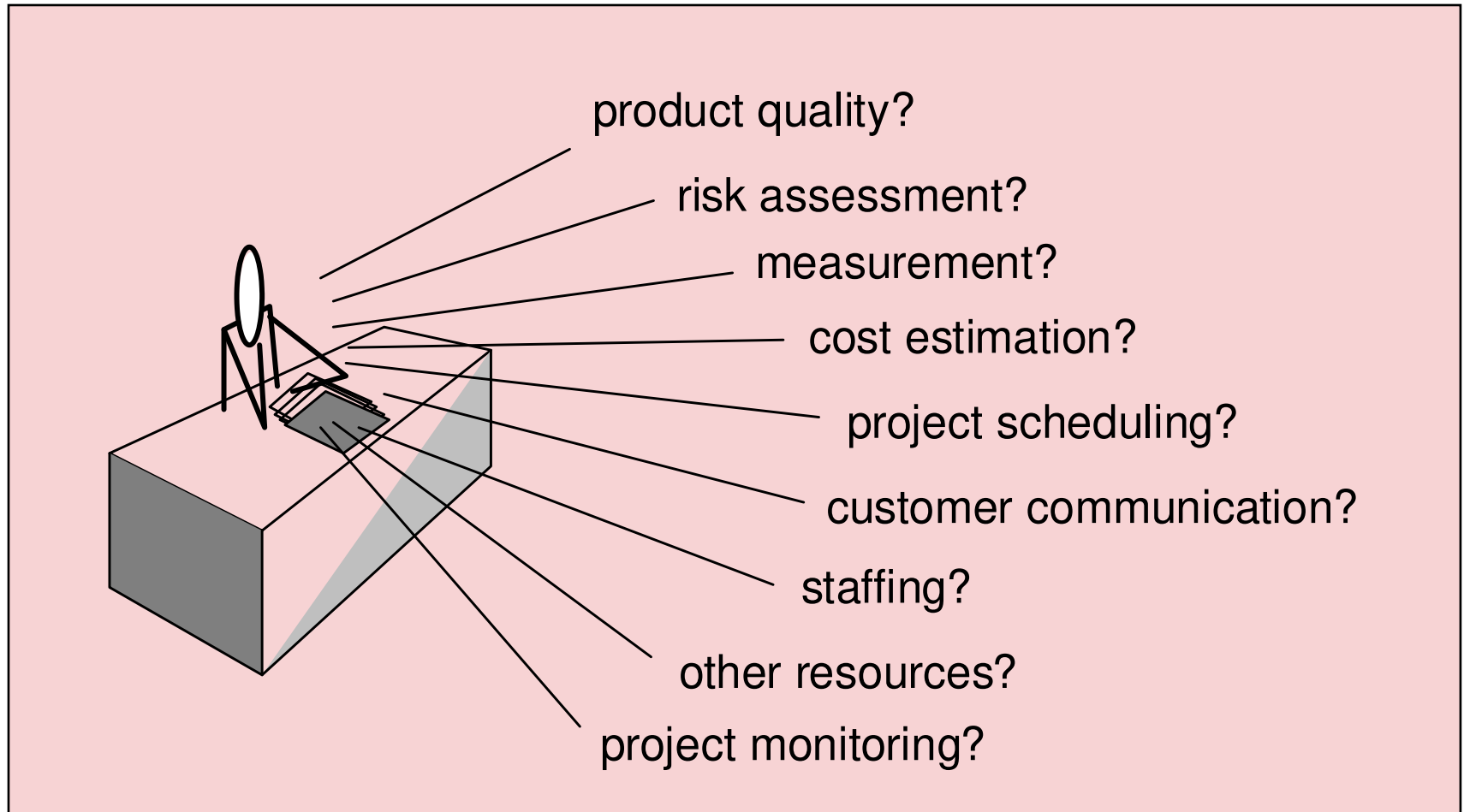
Management functions can be categorized as

1. planning
2. organizing
3. staffing
4. directing
5. controlling

Project Management!!

Project management is a system of management procedures, practices, technologies, skills, and experience that are necessary to successfully manage a project.

Project Management Concerns



Project Management

- Software Projects – factors that influence results:
 - size
 - delivery deadline
 - budgets and costs
 - application domain
 - technology to be implemented
 - system constraints
 - user requirements
 - available resources

Project Management

- Project management is accomplished through the appropriate application and integration of the 47 logically grouped project management processes categorized into 5 process groups:
 - •Initiation
 - •Planning
 - •Execution
 - •Monitoring and control.
 - •Closure

Management Spectrum - The Four P's

- **People** — the most important element of a successful project
- **Product** — the software to be built
- **Process** — the set of framework activities and software engineering tasks to get the job done
- **Project** — all work required to make the product a reality

Stakeholders

- *Senior managers* who define the business issues that often have significant influence on the project.
- *Project (technical) managers* who must plan, motivate, organize, and control the practitioners who do software work.
- *Practitioners* who deliver the technical skills that are necessary to engineer a product or application.
- *Customers* who specify the requirements for the software to be engineered and other stakeholders who have a peripheral interest in the outcome.
- *End-users* who interact with the software once it is released for production use.

Software Teams

How to lead?

How to organize?

How to collaborate?



How to motivate?

How to create good ideas?

Team Leader

- The MOI Model
 - **Motivation.** The ability to encourage (by “push or pull”) technical people to produce to their best ability.
 - **Organization.** The ability to mold existing processes (or invent new ones) that will enable the initial concept to be translated into a final product.
 - **Ideas or innovation.** The ability to encourage people to create and feel creative even when they must work within bounds established for a particular software product or application.

Software Teams

The following factors must be considered when selecting a software project team structure ...

- the **difficulty of the problem** to be solved
- the **size of the resultant program(s)** in lines of code or function points
- the **time that the team will stay together** (team lifetime)
- the **degree to which the problem can be modularized**
- the **required quality and reliability** of the system to be built
- the **rigidity of the delivery date**
- the **degree of sociability** (communication) required for the project

Organizational Paradigms

- **closed paradigm**—structures a team along a traditional hierarchy of authority
- **random paradigm**—structures a team loosely and depends on individual initiative of the team members
- **open paradigm**—attempts to structure a team in a manner that achieves some of the controls associated with the closed paradigm but also much of the innovation that occurs when using the random paradigm
- **synchronous paradigm**—relies on the natural compartmentalization of a problem and organizes team members to work on pieces of the problem with little active communication among themselves

suggested by Constantine [Con93]

Avoid Team “Toxicity”

- A frenzied work atmosphere in which team members waste energy and lose focus on the objectives of the work to be performed.
- High frustration caused by personal, business, or technological factors that cause friction among team members.
- “Fragmented or poorly coordinated procedures” or a poorly defined or improperly chosen process model that becomes a roadblock to accomplishment.
- Unclear definition of roles resulting in a lack of accountability and resultant finger-pointing.
- “Continuous and repeated exposure to failure” that leads to a loss of confidence and a lowering of morale.

Agile Teams

- Team members must have trust in one another.
- The distribution of skills must be appropriate to the problem.
- Mavericks may have to be excluded from the team, if team cohesiveness is to be maintained.
- Team is “self-organizing”
 - An adaptive team structure
 - Uses elements of Constantine’s random, open, and synchronous paradigms
 - Significant autonomy

Team Coordination & Communication

- *Formal, impersonal approaches* include software engineering documents and work products (including source code), technical memos, project milestones, schedules, and project control tools, change requests and related documentation, error tracking reports, and repository data.
- *Formal, interpersonal procedures* focus on quality assurance activities applied to software engineering work products. These include status review meetings and design and code inspections.
- *Informal, interpersonal procedures* include group meetings for information dissemination and problem solving and “collocation of requirements and development staff.”
- *Electronic communication* encompasses electronic mail, electronic bulletin boards, and by extension, video-based conferencing systems.
- *Interpersonal networking* includes informal discussions with team members and those outside the project who may have experience or insight that can assist team members.

The Product Scope

- Scope
 - **Context.** How does the software to be built fit into a larger system, product, or business context and what constraints are imposed as a result of the context?
 - **Information objectives.** What customer-visible data objects are produced as output from the software? What data objects are required for input?
 - **Function and performance.** What function does the software perform to transform input data into output? Are any special performance characteristics to be addressed?
- Software project scope must be unambiguous and understandable at the management and technical levels.

Problem Decomposition

- Sometimes called *partitioning* or *problem elaboration*
- Once scope is defined ...
 - It is decomposed into basic functions
 - It is decomposed into user-visible data objects*or*
 - It is decomposed into a set of problem classes
- Decomposition process continues until all functions or problem classes have been defined

The Process

- Once a process framework has been established
 - Consider project characteristics
 - Determine the degree of rigor required
 - Define a task set for each software engineering activity
 - Task set =
 - Software engineering tasks
 - Work products
 - Quality assurance points
 - Milestones

The Project

- *Projects get into trouble when ...*
 - Software people don't understand their customer's needs.
 - The product scope is poorly defined.
 - Changes are managed poorly.
 - The chosen technology changes.
 - Business needs change [or are ill-defined].
 - Deadlines are unrealistic.
 - Users are resistant.
 - Sponsorship is lost [or was never properly obtained].
 - The project team lacks people with appropriate skills.
 - Managers [and practitioners] avoid best practices and lessons learned.

Common-Sense Approach to Projects

- *Start on the right foot.* This is accomplished by working hard (very hard) to understand the problem that is to be solved and then setting realistic objectives and expectations.
- *Maintain momentum.* The project manager must provide incentives to keep turnover of personnel to an absolute minimum, the team should emphasize quality in every task it performs, and senior management should do everything possible to stay out of the team's way.
- *Track progress.* For a software project, progress is tracked as work products (e.g., models, source code, sets of test cases) are produced and approved (using formal technical reviews) as part of a quality assurance activity.
- *Make smart decisions.* In essence, the decisions of the project manager and the software team should be to “keep it simple.”
- *Conduct a postmortem analysis.* Establish a consistent mechanism for extracting lessons learned for each project.

To Get to the Essence of a Project

- Why is the system being developed?
- What will be done?
- When will it be accomplished?
- Who is responsible?
- Where are they organizationally located?
- How will the job be done technically and managerially?
- How much of each resource (e.g., people, software, tools, database) will be needed?

Barry Boehm [Boe96]

Critical Practices

- Formal risk management
- Empirical cost and schedule estimation
- Metrics-based project management
- Earned value tracking
- Defect tracking against quality targets
- People aware project management

Metrics for the requirements model

- Technical work in software engineering begins with the creation of the requirements model.
- It is at this stage that requirements are derived and a foundation for design is established. Therefore, product metrics that provide insight into the quality of the analysis model are desirable.
- Although relatively few analysis and specification metrics have appeared in the literature, it is possible to adapt metrics that are often used for project estimation and apply them in this context.
- These metrics examine the requirements model with the intent of predicting the “size” of the resultant system.
- Size is sometimes (but not always) an indicator of design complexity and is almost always an indicator of increased coding, integration, and testing effort.

Function Based metrics

- The function point (FP) metric can be used effectively as a means for measuring the functionality delivered by a system.
- Using historical data, the FP metric can then be used to
 - estimate the cost or effort required to design, code, and test the software;
 - predict the number of errors that will be encountered during testing
 - forecast the number of components and/or the number of
 - projected source lines in the implemented system.
- Function points are derived using an empirical relationship based on countable (direct) measures of software's information domain and qualitative assessments of software complexity.

Function Based metrics

- Information domain values are defined in the following manner:
- **Number of external inputs (EIs).** Each external input originates from a user or is transmitted from another application and provides distinct application-oriented data or control information. Inputs are often used to update internal logical files (ILFs). Inputs should be distinguished from inquiries, which are counted separately.
- **Number of external outputs (EOs).** Each external output is derived data within the application that provides information to the user. In this context external output refers to reports, screens, error messages, etc. Individual data items within a report are not counted separately.
- **Number of external inquiries (EQs).** An external inquiry is defined as an online input that results in the generation of some immediate software response in the form of an online output (often retrieved from an ILF).
- **Number of internal logical files (ILFs).** Each internal logical file is a logical grouping of data that resides within the application's boundary and is maintained via external inputs.
- **Number of external interface files (EIFs).** Each external interface file is a logical grouping of data that resides external to the application but provides information that may be of use to the application.

Function Based metrics

- Once these data have been collected, the table in Figure 23.1 is completed and a complexity value is associated with each count.
- Organizations that use function point methods develop criteria for determining whether a particular entry is simple, average, or complex.
- Nonetheless, the determination of complexity is somewhat subjective.
- To compute function points (FP), the following relationship is used:

$$FP = \text{count total} \times [0.65 + 0.01 \times \sum(F_i)]$$

- where count total is the sum of all FP entries obtained from Figure 23.1.

Function Based metrics

FIGURE 23.1

Computing function points	Information Domain Value	Count	Weighting factor				
			Simple	Average	Complex		
	External Inputs (EIs)		×	3	4	6	=
	External Outputs (EOs)		×	4	5	7	=
	External Inquiries (EQs)		×	3	4	6	=
	Internal Logical Files (ILFs)		×	7	10	15	=
	External Interface Files (EIFs)		×	5	7	10	=
	Count total						

Function Based metrics

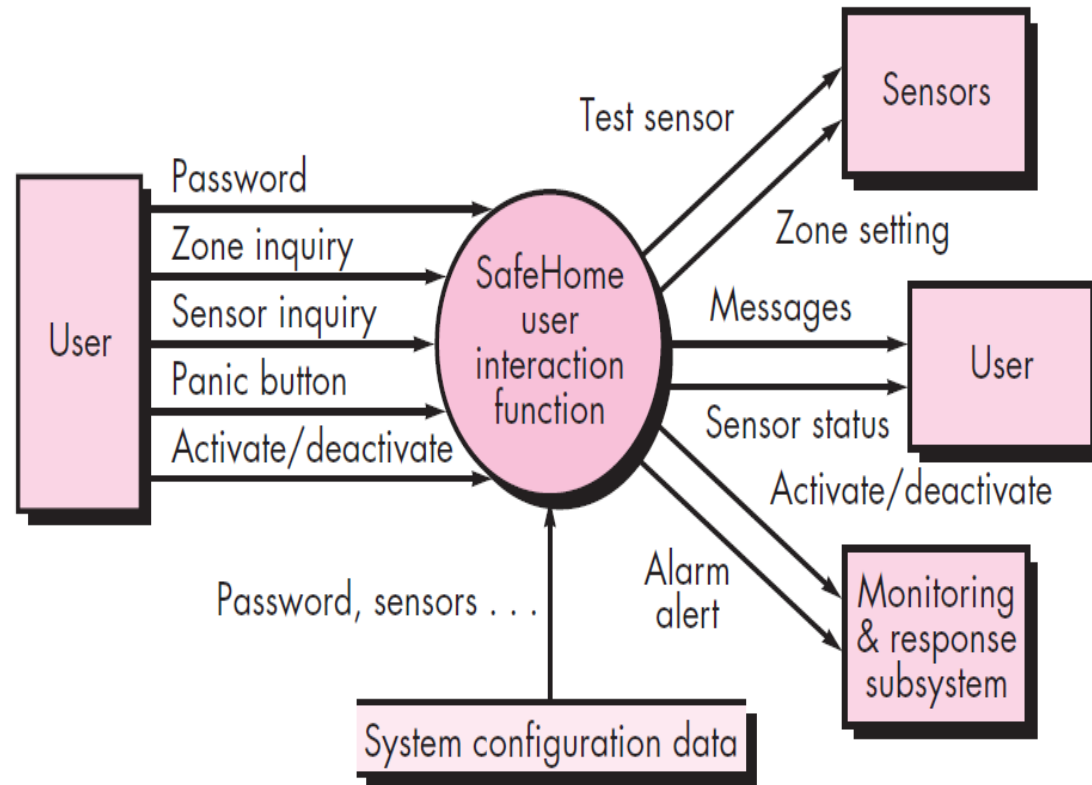
- The F_i ($i = 1$ to 14) are value adjustment factors (VAF) based on responses to the following questions [Lon02]:
 1. Does the system require reliable backup and recovery?
 2. Are specialized data communications required to transfer information to or from the application?
 3. Are there distributed processing functions?
 4. Is performance critical?
 5. Will the system run in an existing, heavily utilized operational environment?
 6. Does the system require online data entry?
 7. Does the online data entry require the input transaction to be built over multiple screens or operations?
 8. Are the ILFs updated online?
 9. Are the inputs, outputs, files, or inquiries complex?
 10. Is the internal processing complex?
 11. Is the code designed to be reusable?
 12. Are conversion and installation included in the design?
 13. Is the system designed for multiple installations in different organizations?
 14. Is the application designed to facilitate change and ease of use by the user?
- Each of these questions is answered using a scale that ranges from 0 (not important or applicable) to 5 (absolutely essential). The constant values in Equation (23.1) and the weighting factors that are applied to information domain counts are determined empirically.

Function Based metrics

Example: SafeHoME Software

FIGURE 23.2

A data flow
model for
SafeHome
software



Function Based metrics

Example: SafeHoME Software

- The function manages user interaction, accepting a user password to activate or deactivate the system, and allows inquiries on the status of security zones and various security sensors.
- The function displays a series of prompting messages and sends appropriate control signals to various components of the security system.
- The data flow diagram is evaluated to determine a set of key information domain measures required for computation of the function point metric.
 - Three external inputs—**password**, **panic button**, and **activate/deactivate**—are shown in the figure along with
 - two external inquiries—**zone inquiry** and **sensor inquiry**.
 - One ILF (**system configuration file**) is shown.
 - Two external outputs (**messages** and **sensor status**) and
 - four EIFs (**test sensor**, **zone setting**, **activate/deactivate**, and **alarm alert**) are also present.
- These data, along with the appropriate complexity, are shown in Figure 23.3

Function Based metrics

Example: SafeHoME Software

FIGURE 23.3

Computing function points	Information Domain Value	Count	Weighting factor			
			Simple	Average	Complex	
External Inputs (EIs)		3	×	3	4	6 = 9
External Outputs (EOs)		2	×	4	5	7 = 8
External Inquiries (EQs)		2	×	3	4	6 = 6
Internal Logical Files (ILFs)		1	×	7	10	15 = 7
External Interface Files (EIFs)		4	×	5	7	10 = 20
Count total						50

Function Based metrics

Example: SafeHoME Software

- The count total shown in Figure 23.3 must be adjusted using Equation (23.1).
- For the purposes of this example, we assume that $\sum(F_i)$ is 46 (a moderately complex product). Therefore,

$$FP = 50 \times [0.65 + (0.01 \times 46)] = 56$$

- Based on the projected FP value derived from the requirements model, the project team can estimate the overall implemented size of the SafeHome user interaction function.
- Assume that past data indicates that one FP translates into 60 lines of code (an object-oriented language is to be used) and that 12 FPs are produced for each person-month of effort.
- These historical data provide the project manager with important planning information that is based on the requirements model rather than preliminary estimates.
- Assume further that past projects have found an average of three errors per function point during requirements and design reviews and four errors per function point during unit and integration testing.
- These data can ultimately help you assess the completeness of your review and testing activities.

Cocomo Model

- In his classic book on “software engineering economics,” Barry Boehm [Boe81] introduced a hierarchy of software estimation models bearing the name COCOMO
 - COnstructive COst MOdel.
- The original COCOMO model became one of the most widely used and discussed software cost estimation models in the industry.
- It has evolved into a more comprehensive estimation model, called COCOMOII [Boe00].

Cocomo Model

- COCOMO applies to three classes of software projects:
 1. **Organic projects** - "small" teams with "good" experience working with "less than rigid" requirements
 2. **Semi-detached projects** - "medium" teams with mixed experience working with a mix of rigid and less than rigid requirements
 3. **Embedded projects** - developed within a set of "tight" constraints (hardware, software, operational,)
- The basic COCOMO equations take the form
 - **Effort Applied (E)** = $a_b(KLOC)^{b_b}$ [**man-months**]
 - **Development Time (TDEV)** = $c_b(\text{Effort Applied})^{d_b}$ [months]
 - **People required (P)** = Effort Applied / Development Time [count]

Cocomo I / Basic Cocomo / Cocomo81 FP based

- Another approach that can be adopted using Basic COCOMO is to convert it to Function Point based.
- Using following Steps:

1. Compute Count total by any one of the methods based on the given data.

FIGURE 23.1

Computing function points	Information Domain Value	Count	Weighting factor				
			Simple	Average	Complex		
	External Inputs (EIs)	×	3	4	6	=	
	External Outputs (EOs)	×	4	5	7	=	
	External Inquiries (EQs)	×	3	4	6	=	
	Internal Logical Files (ILFs)	×	7	10	15	=	
	External Interface Files (EIFs)	×	5	7	10	=	
Count total		→					

Cocomo I / Basic Cocomo / Cocomo81

FP based

2. Compute Adjustment Values

Factor	Value
Backup and recovery	4
Data communications	2
Distributed processing	0
Performance critical	4
Existing operating environment	3
Online data entry	4
Input transaction over multiple screens	5
Master files updated online	3
Information domain values complex	5
Internal processing complex	5
Code designed for reuse	4
Conversion/installation in design	3
Multiple installations	5
Application designed for change	5
Value adjustment factor	1.17

Cocomo I / Basic Cocomo / Cocomo81

FP based

3. Compute FP

$$FP_{est} = \text{count total} \times [0.65 + 0.01 \times \sum(F_i)]$$

4. Choose a programming language for calculation of LOC per FP based on historical average no of lines for 01 FP.
- $(LOC_{avg} * FP)$

Programming Language	LOC per Function Point			
	Average	Median	Low	High
Access	35	38	15	47
Ada	154	—	104	205
APS	86	83	20	184
ASP 69	62	—	32	127
Assembler	337	315	91	694
C	162	109	33	704
C++	66	53	29	178
Clipper	38	39	27	70
COBOL	77	77	14	400
Cool:Gen/IEF	38	31	10	180
Culprit	51	—	—	—
DBase IV	52	—	—	—
Easytrieve+	33	34	25	41
Excel47	46	—	31	63
Focus	43	42	32	56
FORTRAN	—	—	—	—
FoxPro	32	35	25	35
Ideal	66	52	34	203
IEF/Cool:Gen	38	31	10	180
Informix	42	31	24	57
Java	63	53	77	—
JavaScript	58	63	42	75

Cocomo I / Basic Cocomo / Cocomo81

FP based

5. Calculate using Basic cocomo by using obtained LOC and selecting the class of software project

Software project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

OR

Software project	Effort	Schedule
Organic	$E = 2.4 * (KLOC)^{1.05}$	$TDEV = 2.5 * (E)^{0.38}$
Semi-detached	$E = 3.0 * (KLOC)^{1.12}$	$TDEV = 2.5 * (E)^{0.35}$
Embedded	$E = 3.6 * (KLOC)^{1.20}$	$TDEV = 2.5 * (E)^{0.32}$

Cocomo I / Cocomo81 FP based

Problem: A software (Free Online Pdf Converter), having an estimated average weighting factor. The software has EIs = 4, EOs=3, EQs=2, ILFs=2, EIFs=4. Supposing that the software is being developed with a set of tight constraints, calculate effort, development time and average staffing using CoCoMo81.

1. Calculate the Size in Function Points (FP)

- $UFP = (EIs \times 4) + (EOs \times 5) + (EQs \times 4) + (ILFs \times 10) + (EIFs \times 7)$
- $UFP = (4 \times 4) + (3 \times 5) + (2 \times 4) + (2 \times 10) + (4 \times 7) = 16 + 15 + 8 + 20 + 28 = 87 \text{ FP}$
- Assuming a typical **weighting factor (value adjustment factor)** of 1.0 (since none is provided), the **Adjusted Function Points (AFP) = 87 FP**.

Cocomo I / Cocomo81 FP based

2. Convert Function Points to Source Lines of Code (SLOC)

- For a **high-level language like Java or C#**, we can use the conversion factor:
- $1 \text{ FP} \approx 46 \text{ SLOC}$ (for average complexity)
- $\text{SLOC} = 87 \times 46 = 4002 \approx 4\text{KLOC}$

3. Apply COCOMO 81 Effort Equation

- For **Embedded Mode (small, simple projects with tight constraints)**, the coefficients are:
- $a=3.6$ $b=1.20$
- The **Effort (E)** in **Person-Months (PM)** is:
- $E = a \times (\text{KLOC})^b$
- $E = 3.6 \times (4.0)^{1.20} = 19 \text{ PM}$ (19 person-months)

Cocomo I / Cocomo81 FP based

4. Calculate Development Time (T)

- For **Embedded Mode**, the coefficients are:
- $c=2.5$ $d=0.32$
- $T=c \times (E)^T = 2.5 \times (19)^{0.32} = 6.41$ months (6.41 months)

5. Compute Average Staffing (AvgStaff)

- $\text{AvgStaff} = E/T$
- $\text{AvgStaff} = 19/6.41 = 2.96$ (3 developers)